CS5486 Assignment 1

October 8, 2026

1 CS5486 Assignment 1

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, HTML

%matplotlib inline
plt.rcParams.update({"figure.dpi": 110, "font.size": 10})

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
XB = 2 * X - 1
t_or = np.array([-1, 1, 1, 1], dtype=float)
t_xor = np.array([-1, 1, 1, -1], dtype=float)
t01 = np.array([0, 1, 1, 0], dtype=float)

def show_table(headers, rows):
    header = "".join(f"<th>{v}</th>" for v in headers)
    body = "".join("<tr>" + "".join(f"<td>{v}</td>" for v in row)
                   + "</tr>" for row in rows)
    display(HTML(f"<table><tr>{header}</tr>{body}</table>"))

show_table(["x1", "x2", "OR", "XOR"],
           [[0, 0, 0, 0], [0, 1, 1, 1], [1, 0, 1, 1], [1, 1, 1, 0]])
```

```
<IPython.core.display.HTML object>
```

1.1 1. Perceptron and ADALINE

The bipolar threshold is $g(z) = +1$ for $z \geq 0$, and -1 otherwise. With bias included in θ and $q = [x_1, x_2, 1]$, the perceptron updates $\theta \leftarrow \theta + \eta(t - g(\theta^T q))q$ after each sample. ADALINE instead trains on the **continuous linear output**, using $\theta \leftarrow \theta - \eta A^T(A\theta - t)/4$ once per epoch.

Both use learning rate **0.1**. The perceptron starts at zero and runs for **100 epochs**; ADALINE starts at $(0.2, -0.1, 0.4)$ and runs for **200 epochs**. The sample order is fixed. An epoch processes all four samples; epoch 0 is initialization. Fixed horizons retain the curves after classification becomes correct.

[2]:
```
def bipolar(z):
    return np.where(z >= 0, 1, -1)  # sign(0) must be +1 here.

def perceptron(X, t, lr=0.1, epochs=100):
    A = np.c_[X, np.ones(len(X))]
    theta = np.zeros(A.shape[1])
    online = []
    errors = [np.sum(bipolar(A @ theta) != t)]
    for _ in range(epochs):
        mistakes = 0
        for q, target in zip(A, t):
            y = bipolar(q @ theta)
            mistakes += int(y != target)
            theta += lr * (target - y) * q
        online.append(mistakes)
        errors.append(np.sum(bipolar(A @ theta) != t))
    return {"scores": A @ theta, "online": online, "errors": errors}

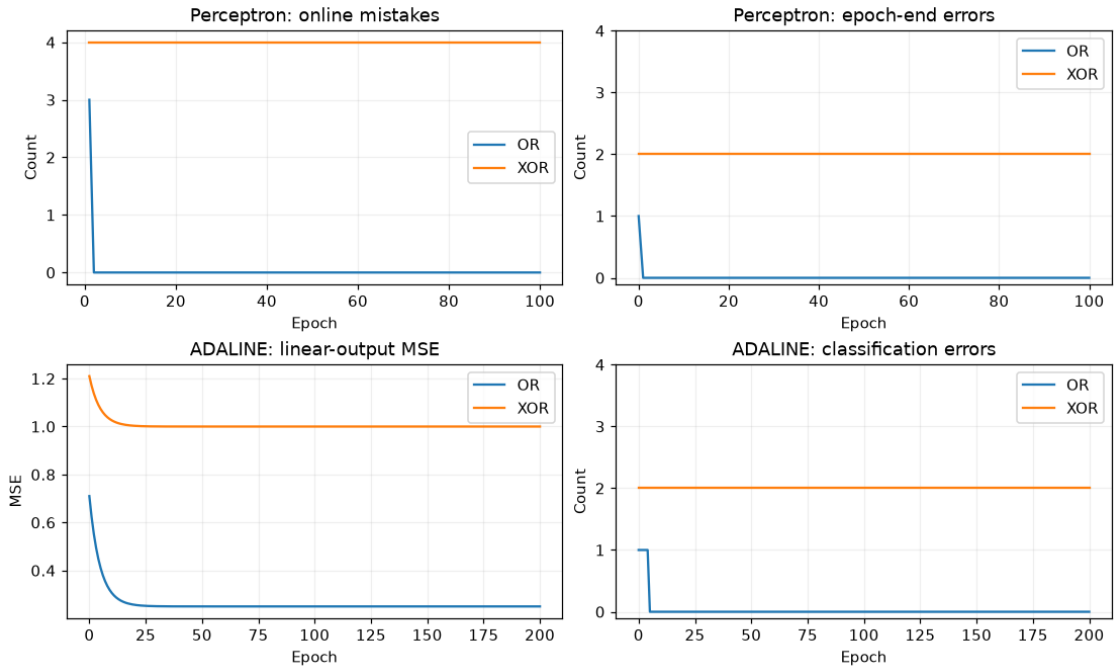
def linear_gd(A, t, theta, lr, epochs, threshold=0):
    # Shared least-squares training for ADALINE and the RBF output layer.
    theta = np.array(theta, dtype=float)
    mse, errors = [], []
    for epoch in range(epochs + 1):
        y = A @ theta
        mse.append(np.mean((y - t) ** 2))
        errors.append(np.sum((y >= threshold) != (t > 0)))
        if epoch < epochs:
            theta -= lr * A.T @ (y - t) / len(t)
    return {"scores": y, "mse": mse, "errors": errors, "theta": theta}
```

[3]:
```
A = np.c_[XB, np.ones(4)]
per = {name: perceptron(XB, t) for name, t in [("OR", t_or), ("XOR", t_xor)]}
ada = {name: linear_gd(A, t, [0.2, -0.1, 0.4], 0.1, 200)
       for name, t in [("OR", t_or), ("XOR", t_xor)]}

fig, axes = plt.subplots(2, 2, figsize=(10, 6), constrained_layout=True)
for name, color in [("OR", "tab:blue"), ("XOR", "tab:orange")]:
    axes[0, 0].plot(range(1, 101), per[name]["online"], label=name, color=color)
    axes[0, 1].plot(per[name]["errors"], label=name, color=color)
    axes[1, 0].plot(ada[name]["mse"], label=name, color=color)
    axes[1, 1].plot(ada[name]["errors"], label=name, color=color)
titles = ["Perceptron: online mistakes", "Perceptron: epoch-end errors",
          "ADALINE: linear-output MSE", "ADALINE: classification errors"]
for i, ax in enumerate(axes.flat):
    ax.set(title=titles[i], xlabel="Epoch", ylabel="MSE" if i == 2 else "Count")
    if i != 2:
        ax.set_yticks(range(5))
```

```
    ax.grid(alpha=0.2)
    ax.legend()
plt.show()

rows = []
for model, runs in [("Perceptron", per), ("ADALINE", ada)]:
    for name, r in runs.items():
        mse = f"{r['mse'][-1]:.4f}" if "mse" in r else "Not optimized"
        accuracy = f"{100 * (1 - r['errors'][-1] / 4):.0f}%"
        rows.append([model, name, bipolar(r["scores"]).tolist(), mse, accuracy])
show_table(["Model", "Task", "Predictions", "Final MSE", "Training accuracy"],␣
 ↪rows)
```




```
<IPython.core.display.HTML object>
```

Results. Both models classify OR correctly. The perceptron has correct epoch-end predictions after epoch 1 and no online mistakes from epoch 2. On XOR it keeps cycling: four online mistakes per epoch, but two errors when evaluated using the epoch-end weights. OR is linearly separable; XOR is not.

ADALINE reaches MSE **0.25 on OR** while classifying every sample correctly. On XOR, its MSE converges to **1**, but the linear outputs approach zero and do not solve the classification task. Thus loss convergence does not imply correct classification. The reported 50% XOR accuracy is specific to these runs, not an upper bound for all linear classifiers.

1.2 2. MLP and Gaussian RBF on XOR

MLP: a **2-2-1** network with unipolar sigmoid $\sigma(z) = 1/(1+e^{-z})$ in both hidden and output layers. Full-batch backpropagation uses learning rate **1.0** for **20,000 epochs**. Seed **1** initializes weights from $N(0,1)$ and biases from $N(0, 0.2^2)$.

RBF: two fixed Gaussian centers $c_1 = (1,1)$ and $c_2 = (0,0)$, with $\phi_j(x) = \exp(-\|x - c_j\|^2)$, equivalent to width $\sigma = 1/\sqrt{2}$. The output is $y = v_1\phi_1 + v_2\phi_2 + b$. Only output weights and bias are trained, starting at zero, with learning rate **0.5** for **2,000 epochs**. Both networks predict class 1 when their continuous output is at least **0.5**.

[4]:
```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

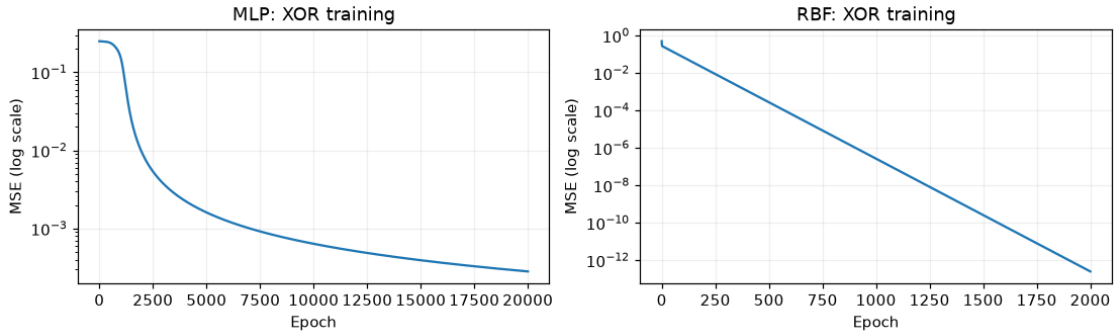
def mlp(X, t, lr=1.0, epochs=20000):
    rng = np.random.default_rng(1)
    W1 = rng.normal(0, 1, (2, 2))
    b1 = rng.normal(0, 0.2, (1, 2))
    W2 = rng.normal(0, 1, (2, 1))
    b2 = rng.normal(0, 0.2, (1, 1))
    T = t[:, None]
    mse = []
    for epoch in range(epochs + 1):
        H = sigmoid(X @ W1 + b1)
        Y = sigmoid(H @ W2 + b2)
        mse.append(np.mean((Y - T) ** 2))
        if epoch < epochs:
            d2 = (Y - T) * Y * (1 - Y) / len(X)
            d1 = (d2 @ W2.T) * H * (1 - H)
            # Compute both error signals before changing the weights.
            W1 -= lr * X.T @ d1
            b1 -= lr * d1.sum(axis=0, keepdims=True)
            W2 -= lr * H.T @ d2
            b2 -= lr * d2.sum(axis=0, keepdims=True)
    return {"scores": Y.ravel(), "mse": mse}

m = mlp(X, t01)
centers = np.array([[1, 1], [0, 0]])
Phi = np.exp(-np.sum((X[:, None, :] - centers[None, :, :]) ** 2, axis=2))
B = np.c_[Phi, np.ones(4)]
r = linear_gd(B, t01, [0, 0, 0], lr=0.5, epochs=2000, threshold=0.5)
```

[5]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 3), constrained_layout=True)
for ax, name, result in zip(axes, ["MLP", "RBF"], [m, r]):
    ax.semilogy(result["mse"])
    ax.set(title=f"{name}: XOR training", xlabel="Epoch", ylabel="MSE (log␣
 ↪scale)")
    ax.grid(alpha=0.2)
```

```
plt.show()

rows = []
for i, (x1, x2) in enumerate(X.astype(int)):
    rows.append([f"({x1}, {x2})", int(t01[i]), f"{m['scores'][i]:.8f}",
                 int(m["scores"][i] >= 0.5), f"{r['scores'][i]:.9g}",
                 int(r["scores"][i] >= 0.5)])
show_table(["Input", "Target", "MLP output", "MLP class", "RBF output", "RBF␣
 ↪class"], rows)
show_table(["Model", "Final MSE", "Training accuracy"],
           [[name, f"{v['mse'][-1]:.6g}",
             f"{100 * np.mean((v['scores'] >= 0.5) == t01):.0f}%"]
            for name, v in [("MLP", m), ("RBF", r)]])
```




```
<IPython.core.display.HTML object>

<IPython.core.display.HTML object>
```

Conclusion. Both MLP and RBF classify all four XOR samples correctly. Their nonlinear hidden features overcome the limitation of the single-layer models. The MLP learns its hidden features, while this RBF uses fixed Gaussian features. The different epoch counts do not establish a general speed ranking, and MLP convergence can depend on initialization.